

Trabajo Práctico Final
Programación I

1er Cuatrimestre 2026

Integrantes:

Jaimes Valentina - 45.623.624

Jara Rodas Valentina - 47.220.213

Vasquez Ludmila - 44.837.742

Emails:

valentinajaimes003@gmail.com
ludmilavasquez08@gmail.com
valentinajararodas@gmail.com

Grupo 06 - Comisión 05

1 .Introducción

En el presente trabajo se llevará a cabo el desarrollo del videojuego *“Super Elizabeth Sis”* a partir de los contenidos trabajados en la materia. Nuestro objetivo principal es poder crear el juego trabajando sobre un entorno dado, en donde nuestro personaje principal Elizabeth pueda avanzar sobre el escenario, enfrentando enemigos para llegar con vida al castillo. Para ello, aplicaremos nuestros conocimientos sobre el manejo de objetos con sus respectivas clases, constructores y métodos, y el trabajo sobre arreglos de objetos.

2. Descripción de clases + Implementación

Class Princesa:

La clase Princesa representa al personaje principal del juego. Se encarga de manejar su movimiento (izquierda, derecha y salto), sus colisiones con el entorno (islas y enemigos), su estado (saltando, cayendo o disparando) y su representación gráfica mediante distintas imágenes según la dirección en la que se mueve.

Variables de instancia:

- Coordenadas de posición (x,y)
- Dimensiones de la imagen y bordes para las colisiones (bordeSuperior, bordeInferior, bordeDerecho, bordeIzquierdo)
- Imágenes de la princesa (prin, prinDer, prinIzq, prinFrente)
- Velocidad, direccion y altura máxima del salto
- Variables de control para los estados de salto, caída y disparo (boolean saltando, boolean cayendo, boolean disparando)

Métodos Principales:

- **Constructor:** Inicializa la posición de la princesa, carga las imágenes y define los valores iniciales para el movimiento y las colisiones.
- **Métodos:**

  dibujar(Entorno entorno): Dibuja la imagen actual de la princesa en pantalla

  mover(char direccion): Permite desplazar a la princesa hacia la izquierda o hacia la derecha (actualizando la imagen asignada a cada posición).

  saltar(Mapa mapa): Controla el salto de la princesa teniendo en cuenta las colisiones y límites del salto.

caer(Mapa mapa): Aplica la gravedad cuando la princesa no está apoyada en ninguna isla.

colisionPieIsla(Mapa mapa): Se encarga de verificar si la princesa está sobre alguna isla.

colisionCabezaIsla(Mapa mapa): Detecta las colisiones durante el salto con la parte inferior de una isla.

actualizarBordes(): Calcula los bordes de colisión según en qué posición actual se encuentre el personaje.

Problemas y Soluciones:

Uno de los problemas que surgió fue lograr que la princesa salte de forma adecuada entre las plataformas, ya que durante las pruebas del juego la princesa podía atravesar algunas islas o no caía correctamente cuando correspondía. Para resolverlo, agregamos controles de colisión y se utilizaron las variables saltando y cayendo para manejar mejor cada movimiento.

```
if(entorno.sePresiono(entorno.TECLA_ARRIBA)&& !princesa.saltando && !princesa.cayendo) {
                princesa.yy=princesa.y;
                princesa.saltando=true;
        }   //con esto pudimos resolver el problema del doble salto
```

Class Isla:

La clase Isla va a representar cada una de las plataformas sobre las que podrá caminar la Princesa. En esta clase definiremos su posición en pantalla, bordes e imagen.

Variables de Instancia:

- Coordenadas del centro de la imagen: X e Y
- Medidas para la escala: Ancho y Alto
- Bordes: bordeSuperior, bordeInferior, bordeIzquierdo, bordeDerecho
- Imágenes: Image isla, nombreIsla → nombre del archivo para cargar las imágenes.

Métodos Principales:

- **Constructor:** Recibe como parámetro la posición donde debe crearse la isla y el nombre de la imagen que usará (X, Y, nombre). Luego, carga el archivo de la imágen, calcula la escala, y define los bordes.
- **Métodos:**

dibujar(Entorno e): Recibe el entorno (e) y dibuja la imagen de la isla con su coordenada y escala correspondiente.

```
public void dibujar(Entorno e) {
        e.dibujarImagen(this.isla, this.x, this.y, 0, this.escala); }
```

Class Mapa:

La clase Mapa se encargará de generar y controlar gran parte del entorno por donde la Princesa deberá jugar. Dicho entorno tendrá movimiento y creará las plataformas (Islas) de manera aleatoria en cada juego nuevo.

Variables de Instancia:

- Arreglo de objetos Isla[], donde se guardarán las plataformas que se vayan creando
- Cantidad de islas en mapa (tamanio → cant. max. de islas en mapa / islasEnMapa → contador de cuántas islas tenemos en el arreglo)
- Coordenadas para islas (X, Y)
- Random para islas (random)
- Imagen del Castillo y coordenadas (castillo, castilloX, castilloY)

Métodos Principales:

- **Constructor:** Primero definimos el tamaño del arreglo e inicializamos un bucle for en donde iremos construyendo las islas de manera aleatoria. Para ello, tendremos en cuenta la posición en la que aparecerán las islas en cada nivel (y=500, entonces estará abajo, y=350 en el medio, y=200 nivel superior).
- **Métodos:**

  obtenerPrimeraIsla(): Devuelve la primer isla del arreglo, nos va a servir para poder posicionar a la princesa apenas arranque el nivel.

  dibujar(Entorno e): Recorre el arreglo con un bucle y va dibujando las islas en la pantalla. Además, dibuja también el castillo al final del mapa.

  moverMapa(double dx): Da el movimiento a la pantalla. Recibe como parámetro el valor DX y va sumándolo a la posición de las islas (teniendo en cuenta sus bordes)

Problemas y Soluciones:

El principal problema fue con la creación de las islas en aleatorio, ya que nos ocurría que una isla quedaba en el nivel superior y la otra en el inferior, pero al no tener una en el medio, la Princesa no podía saltar. Como solución rápida, tuvimos que condicionar un patrón tipo escalera, donde si una isla aparece en el nivel de abajo, la siguiente deberá ir al medio, y una vez que se encuentra allí, podrá subir o bajar dependiendo del random (es decir, se va a crear o una isla superior o una inferior).

```
// Defino un patron tipo escalera pero con random
                        if (nivel == 0) {
                                nivelRandom = 1;
                        } else if (nivel == 1) {
                                nivelRandom = r.nextBoolean() ? 2 : 0;
                        } else {
                                nivelRandom = 1;
                        }
```

Class Enemigos:

La clase Enemigos se encarga de representar a los enemigos del juego, su detección de colisiones con las islas del mapa y su interacción con los bordes de la pantalla. Además maneja la imagen del enemigo según la dirección en que se mueve.

Variables de instancia:

-Coordenadas del enemigo(x,y) para determinar su posición en pantalla.
-Dimensiones(ancho,alto),que se calculan a partir de la imagen escalada.
-Bordes de colisión (bordeSuperior, bordeInferior,bordeDerecho,bordeIzquierdo), permiten detectar interacciones con el entorno.
-Velocidad, define qué tan rápido se mueve horizontalmente.
-Escala, determina el tamaño con el que se dibuja la imagen.
-Dirección, indica hacia dónde se mueve(positivo derecha,negativo izquierda).
-Alturamaxima, límite pensado para relacionar el movimiento del enemigo con las plataformas del mapa.
-Entorno, referencia al entorno de dibujo.

Métodos principales:

- ***Constructor:***Inicializa la posición del enemigo, su dirección y sus parámetros de movimiento. Carga las imágenes correspondientes y calcula el ancho y alto del enemigo en base a la escala y al tamaño de la imagen.

- ***Métodos:***

dibujar(Entorno e): Dibuja al enemigo en pantalla usando la imagen actual(segun dirección) y la escala definida.

actualizarBordes(): Calcula los bordes del enemigo en base a su posición y tamaño. Esto es fundamental para las colisiones.

mover(): Actualiza la posición del enemigo sumando la velocidad según la dirección. Además, cambia la imagen del enemigo dependiendo de si se mueve hacia la derecha o izquierda.

*colisionIsla(mapa m):*Recorre el arreglo de islas del mapa y verifica si el enemigo está colisionando con alguna de ellas

*fueraDepantalla():*Controla si el enemigo salió de los límites visibles del juego,lo cual permite eliminarlo o reiniciarlo.

Problemas y soluciones:

El problema que surgió en esta clase fue la detección de colisiones con las islas, ya que el enemigo podía atravesarlas si no se actualizaban correctamente los bordes. La solución fue llamar siempre a *actualizarBordes()* antes de verificar colisiones.

```
public boolean colisionIsla(Mapa m) {
                                actualizarBordes();
                                for(int i = 0; i < m.tamanio; i++) {
                                    //resto del codigo    }
```

Class Proyectil:

La clase Proyectil representa los disparos que se generan en el juego que serán utilizados por la Princesa. En él se desarrollará el control, desplazamiento y destrucción del mismo dentro del entorno.

Variables de Instancia:

- Para la posición y dimensión del disparo. (X, Y, ancho, alto)
- Bordes para colisiones + escala de imagen. (bordeSuperior, bordeInferior, bordeIzquierdo, bordeDerecho / escala)
- Movimiento (velocidad, DX, DY, boolean activo
- Imagen y Entorno (image Proyectil, Entorno e)

Métodos Principales:

- ***Constructor:*** Se ejecuta cuando la Princesa dispara. Va a tomar como parámetros la posición del personaje (xInicial/yInicial) y del mouse

(mouseX/mouseY). Una vez que carga la imagen con su respectiva escala, calcula la dirección del disparo (DX,DY) a través de la función Math.sqrt que calcula la distancia aplicando el módulo entre X e Y. Además, se inicia el disparo como activo.

- **Métodos:**

  mover(): A las coordenadas X e Y, se le suma la velocidad multiplicada por la dirección para el movimiento.

  dibujar(Entorno entorno): Si el disparo está activo, entonces el entorno lo muestra en pantalla.

  fueraDePantalla(): Se fija si el disparo superó los límites de la pantalla definidos (800x600px)

Problemas y Soluciones:

- Nos costó buscar la forma en la que el proyectil viaje calculando la distancia con el 'click'. Si bien quisimos implementar la lógica que vimos de ejemplo con otro proyecto, nuestra solución más práctica fue buscar algún tipo de función ya conocida que pueda realizar ese cálculo de ángulos de forma inmediata.

```
double modulo = Math.sqrt(distanciaX * distanciaX + distanciaY * distanciaY);
dx = distanciaX / modulo; //Dirección del disparo en X
dy = distanciaY / modulo; //Dirección del disparo en Y
actualizarBordes();
```

Class Juego:

Variables de Instancia:

- Objeto Entorno utilizado para ejecutar y mostrar el juego.
- Objeto Princesa, Isla, Mapa y Proyectil.
- Imágenes utilizadas para el fondo, las vidas, pantalla de victoria y derrota
- Arreglo de enemigos
- Variable enemigosMinimos para controlas la cantidad de enemigos que se muestran en pantalla
- Generador de números aleatorios (Random)
- Variable vidas para controlar la cantidad de vidas restantes de la princesa.
- Variables booleanas juegoPerdido y llegoAlFinal para controlar los estados finales de la partida.

Métodos Principales:

- **Constructor:** Inicializa todos los objetos necesarios para el funcionamiento del juego, carga las imágenes y configura el entorno.
- **Métodos:**

  tick(): Es el método principal del juego. Se ejecuta constantemente y controla el movimiento de la princesa, los saltos, la gravedad, la generación de enemigos, los disparos, las colisiones, el sistema de vidas y las condiciones de victoria o derrota.

  enemigosVivos(): Cuenta la cantidad de enemigos que se muestran en pantalla.

  generarEnemigo(): Genera nuevos enemigos de forma aleatoria cuando la cantidad en pantalla es menor a la mínima establecida.

Problemas y Soluciones:

Un problema que surgió fue implementar un sistema de vidas para la princesa, ya que se debía representar visualmente cuántas vidas le quedan al jugador. Como solución agregamos una variable que sea capaz de controlar la cantidad de vidas disponibles y utilizamos imágenes de corazones para mostrarlas en pantalla. De esta manera, cada vez que la princesa es alcanzada por un enemigo o cae fuera del mapa, pierde una vida y desaparece uno de los corazones.

```
//Vidas
for(int i = 0; i < vidas; i++) {
        entorno.dibujarImagen(corazon, 40 + i*40, 30, 0, 0.08);  }
```

3. Conclusión

A lo largo del desarrollo hemos podido poner en práctica muchos de los temas vistos en la materia, como la creación y relación entre clases y objetos, el uso de métodos y arreglos. También aprendimos a resolver problemas que fueron apareciendo durante el desarrollo del juego, especialmente con las colisiones y los movimientos de los personajes.

Aunque hubo algunas dificultades y tuvimos que hacer varias pruebas hasta que todo funcionara correctamente, fue una experiencia muy útil que nos permitió poner en práctica los conocimientos adquiridos y comprender mejor cómo desarrollar un proyecto de programación completo.